

TABLE OF CONTENTS

[illegible]

[illegible]

Ex.No: 01	The Tower of Hanoi using recursion
Date:	

Aim:

To write a program to solve the tower of Hanoi using recursion.

Procedure:

Step 1:Start the Program

Step 2:If n is equal to 0, then return.

Step 3: Call TowerOfHanoi with parameters (n - 1, source, auxiliary, target).

Step 4:Print "Move disk n from source to target".

Step 5:Call TowerOfHanoi with parameters (n - 1, auxiliary, target, source).

Step 6:Read numDisks from the user.

Step 7:Call TowerOfHanoi with parameters (numDisks, 'A', 'C', 'B').

Step 8: Display Result

Step 9:End of the Program

Program:

```
#include <iostream>

void TowerOfHanoi(int n, char source, char target, char auxiliary) {
    if (n > 0) {
        TowerOfHanoi(n - 1, source, auxiliary, target);
        std::cout<< "Move disk " << n << " from " << source << " to " << target << std::endl;
        TowerOfHanoi(n - 1, auxiliary, target, source);
    }
}

int main() {
    int numDisks;
    std::cout<< "Enter the number of disks: ";
    std::cin>>numDisks;
    TowerOfHanoi(numDisks, 'A', 'C', 'B');
    return 0;
}
```

Output:

```
Enter the number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Result:

Thus, the given Program has been executed successfully and output verified.

Ex.No: 02	Binary Search Tree using Traversals
Date:	

Aim:

To Write a program to traverse through binary search tree using traversals.

Procedure:

Step 1: Start the Program

Step 2: Create a new node (temp) with the entered value.

Step 3: Loop indefinitely

- Insert Elements
- Preorder Traversal
- Inorder Traversal
- Postorder Traversal
- Exit

Step 4: Read user's choice (ch)

Step 5: Switch on the user's choice:

- i. Case 1: Call insertElements() function
- ii. Case 2: Call preorder(root) function
- iii. Case 3: Call inorder(root) function
- iv. Case 4: Call postorder(root) function

Step 6: Display Result

Step 7: End of the Program

Program:

```
#include<iostream>
#include<stdlib.h>
using namespace std;
struct st
{
int data;
struct st *left;
struct st *right;
};
struct st *root=NULL,*temp;
void insertElements();
void preorder(struct st *);
void inorder(struct st *);
void postorder(struct st *);
int main()
{
int ch;

while(1)
{
cout<<"\n 1.insert Elements \n 2.preorder \n 3.inorder \n 4.postorder \n 5.exit";
cout<<"\n enter ur choice ";
cin>>ch;
switch(ch)
{
case 1:insertElements();break;
case 2:preorder(root);break;
case 3:inorder(root);break;
case 4:postorder(root);break;
case 5:exit(1);break;
default:cout<<"invalid operation";

}
}
}
void insertElements()
{
struct st *nc,*pNode;
```

```

int v;
cout<<"\n enter the value ";
cin>>v;
temp=new st;
temp->data=v;
temp->left=NULL;
temp->right=NULL;
if(root==NULL)
{
root=temp;
}
else
{
nc=root;
while(nc!=NULL)
{
pNode=nc;
if(v<nc->data)
nc=nc->left;
else
nc=nc->right;
}
if(v<pNode->data)
{
pNode->left=temp;
}
else
pNode->right=temp;
}
}
void preorder(struct st *temp)
{
if(temp!=NULL)
{
cout<<" "<<temp->data;
preorder(temp->left);
preorder(temp->right);
}
}
void inorder(struct st *temp)

```



```

{
    if(temp!=NULL)
    {
        inorder(temp->left);
        cout<<" "<<temp->data;
        inorder(temp->right);
    }
}

void postorder(struct st *temp)
{
    if(temp!=NULL)
    {
        postorder(temp->left);
        postorder(temp->right);
        cout<<" "<<temp->data;
    }
}

```

Output:

```
1.insert Elements
2.preorder
3.inorder
4.postorder
5.exit
enter your choice 2
5 1 3 2 4
```

Result:

Thus, the given Program has been executed successfully and output verified.

Ex.No: 03	Stack using Linked List
Date:	

Aim:

To Write a program to perform various operations on stack using linked list.

Procedure:

Step 1: Start the Program

Step 2: Create Node Class

Step 3: Create Stack Class

Implement the 'push' method to add a new element to the stack.

Implement the 'isEmpty' method to check if the stack is empty.

Implement the 'peek' method to retrieve the top element of the stack.

Implement the 'pop' method to remove the top element from the stack.

Implement the 'display' method to print the elements of the stack.

Step 4: Call s.push(11), s.push(22), s.push(33), s.push(44) to add elements to the stack.

Step 5: Call s.pop() twice to remove elements from the stack

Step 6: Call s.display() to print the updated elements of the stack.

Step 7: Call cout<< "\nTop element is " <<s.peek() <<endl to display the new top element of the stack.

Step 8: End of the Program

Program:

```
#include <bits/stdc++.h>
using namespace std;
class Node {
public:
    int data;
    Node* link;
    Node(int n)
    {
        this->data = n;
        this->link = NULL;
    }
};
class Stack {
    Node* top;
public:
    Stack() { top = NULL; }
    void push(int data)
    {
        Node* temp = new Node(data);
        if (!temp) {
            cout<< "\nStack Overflow";
            exit(1);
        }
        temp->data = data;
        temp->link = top;
        top = temp;
    }
    bool isEmpty()
    {
        return top == NULL;
    }
    int peek()
    {
        if (!isEmpty())
            return top->data;
        else
            exit(1);
    }
};
```

```

    }
    void pop()
    {
        Node* temp;
        if (top == NULL) {
            cout<< "\nStack Underflow" <<endl;
            exit(1);
        }
        else {
            temp = top;
            top = top->link;
            free(temp);
        }
    }
    void display()
    {
        Node* temp;
        if (top == NULL) {
            cout<< "\nStack Underflow";
            exit(1);
        }
        else {
            temp = top;
            while (temp != NULL) {
                cout<< temp->data;
                temp = temp->link;
                if (temp != NULL)
                    cout<< " -> ";
            }
        }
    }
};
int main()
{
    Stack s;
    s.push(11);
    s.push(22);
    s.push(33);
    s.push(44);
    s.display();
}

```

```
cout<< "\nTop element is " <<s.peek() <<endl;
s.pop();
s.pop();
s.display();
cout<< "\nTop element is " <<s.peek() <<endl;

    return 0;
}
```

Output:

44 -> 33 -> 22 -> 11

Top element is 44

22 -> 11

Top element is 22

Result:

Thus, the given Program has been executed successfully and output verified.

Ex.No: 04	OPERATION IN CIRCULAR QUEUE
Date:	

Aim:

To write a program to perform various operation in circular queue.

Procedure:

Step 1: Start the Program

Step 2: Declare a class Circular_Queue.

Step 3: Insert an Element into the Circular Queue

Step 4: Delete an Element from the Circular Queue

Step 5: If front and rear are the same (last element), set front and rear to -1.

Step 6: Declare variables choice and item.

Step 7: Create an instance of Circular_Queue called cq.

Step 8: Take input for the item and call cq.insert(item).

Step 9: Repeat the loop until chooses to quit (choice 4).

Step 10: End of the Program

Program:

```
#include <iostream>
#define MAX 5
using namespace std;
class Circular_Queue
{
    private:
        int *cqueue_arr;
        int front, rear;
    public:
        Circular_Queue()
        {
            cqueue_arr = new int [MAX];
            rear = front = -1;
        }
        void insert(int item)
        {
            if ((front == 0 && rear == MAX-1) || (front == rear+1))
            {
                cout<<"Queue Overflow \n";
                return;
            }
            if (front == -1)
            {
                front = 0;
                rear = 0;
            }
            else
            {
                if (rear == MAX - 1)
                    rear = 0;
                else
                    rear = rear + 1;
            }
            cqueue_arr[rear] = item ;
        }
        void del()
        {
            {
```

```

        if (front == -1)
        {
cout<<"Queue Underflow\n";
return ;
        }
cout<<"Element deleted from queue is : "<<cqueue_arr[front]<<endl;
        if (front == rear)
        {
            front = -1;
            rear = -1;
        }
        else
        {
            if (front == MAX - 1)
                front = 0;
            else
                front = front + 1;
        }
    }
void display()
{
    int front_pos = front, rear_pos = rear;
    if (front == -1)
    {
cout<<"Queue is empty\n";
        return;
    }
cout<<"Queue elements :\n";
    if (front_pos<= rear_pos)
    {
        while (front_pos<= rear_pos)
        {
cout<<cqueue_arr[front_pos]<<" ";
front_pos++;
        }
    }
    else
    {
        while (front_pos<= MAX - 1)
        {

```

```

    cout<<cqueue_arr[front_pos]<<" ";
    front_pos++;
    }
    front_pos = 0;
    while (front_pos<= rear_pos)
    {
    cout<<cqueue_arr[front_pos]<<" ";
    front_pos++;
    }
    }
    cout<<endl;
    }
};
int main()
{
    int choice, item;
    Circular_Queuecq;
    do
    {
    cout<<"1.Insert\n";
    cout<<"2.Delete\n";
    cout<<"3.Display\n";
    cout<<"4.Quit\n";
    cout<<"Enter your choice : ";
    cin>>choice;
    switch(choice)
    {
    case 1:
    cout<<"Input the element for insertion in queue : ";
    cin>>item;
    cq.insert(item);
    break;
    case 2:
    cq.del();
    break;
    case 3:
    cq.display();
    break;
    case 4:
    break;
    }
    }
    while(choice!=4);
}

```

```
    default:  
    cout<<"Wrong choice\n";  
    }  
    }  
while(choice != 4);  
    return 0;  
}
```

Output:

```
1.Insert
2.Delete
3.Display
4.Quit
Enter your choice : 1
Input the element for insertion in queue : 3
```

Result:

Thus, the given Program has been executed successfully and output verified.

Ex.No: 05	SORT AN ARRAY OF AN ELEMENTS USING QUICK SORT
Date:	

Aim:

To write a program to sort an array of an elements using quick sort.

Procedure:

Step 1: Start the Program

Step 2: **swap** function that takes an array and two positions as parameters.

Step 3: Implement Partition Function

Step 4: Implement QuickSort Function

Step 5: Input Array Size and Elements

Step 6: Call QuickSort Function

Step 7: Display Sorted Array

Step 8: End of the program

Program:

```
#include <iostream>
using namespace std;
void swap(int arr[] , int pos1, int pos2){
    int temp;
    temp = arr[pos1];
    arr[pos1] = arr[pos2];
    arr[pos2] = temp;
}
int partition(int arr[], int low, int high, int pivot){
    int i = low;
    int j = low;
    while( i<= high){
        if(arr[i] >pivot){
            i++;
        }
        else{
            swap(arr,i,j);
            i++;
            j++;
        }
    }
    return j-1;
}
void quickSort(int arr[], int low, int high){
    if(low < high){
        int pivot = arr[high];
        int pos = partition(arr, low, high, pivot);

        quickSort(arr, low, pos-1);
        quickSort(arr, pos+1, high);
    }
}
int main()
{
    int n ;
    cout<< " enter the size of array";
    cin>>n;
    int arr[n];
```

```
for( inti = 0 ; i< n; i++){  
    cin>>arr[i];  
}  
quickSort(arr, 0 , n-1);  
cout<<"The sorted array is: ";  
for( inti = 0 ; i< n; i++){  
    cout<<arr[i]<<"\t";  
}  
}
```


Output:

enter the size of array5

32

21

54

76

45

The sorted array is: 21 32 45 54 76

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 06	ASCENDING ORDER USING HEAP SORT
Date:	

Aim:

To write a program to solve number of elements in ascending order using heap sort.

Procedure:

Step 1: Start the Program

Step 2: Initialize heapify function that takes an array, its size n, and an index i.

Step 3: Define a heapSort function that takes an array and its size n.

Step 4: Define Display Function

Step 5: Initialize Array and Display Unsorted Array

Step 6: Call the heapSort function to sort the array.

Step 7: Display the sorted array using the display function.

Step 8: End of the program

Program:

```
#include<iostream>
using namespace std;
void heapify(int arr[], int n, int i)
{
    int largest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;
    if (l < n && arr[l] > arr[largest])
        largest = l;
    if (r < n && arr[r] > arr[largest])
        largest = r;
    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n-1; i >= 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
void display(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout<<arr[i] << "\t";
    }
    cout<< "\n";
}
int main()
{
    int arr[] = {1, 14, 3, 7, 0};
```

```
    int n = sizeof(arr)/sizeof(arr[0]);  
    cout<< "Unsorted array \n";  
    display(arr, n);  
    heapSort(arr, n);  
    cout<< "Sorted array \n";  
    display(arr, n);  
}
```

Output:

Unsorted array

1 14 3 7 0

Sorted array

0 1 3 7 14

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 07	KNAPSACK PROBLEM USING GREEDY METHOD
Date:	

Aim:

To write a program to solve the knapsack problem using greedy method.

Procedure:

Step 1: Start the Program

Step 2: Input function to take user input for the values and weights of items.

Step 3: knapsack function that takes items, the number of items, and the knapsack capacity as parameters.

Step 4: Display the entered data using the display function.

Step 5: Display the maximum value that can be obtained for the given knapsack weight.

Step 6: Display Result

Step 7: End of the Program

Program:

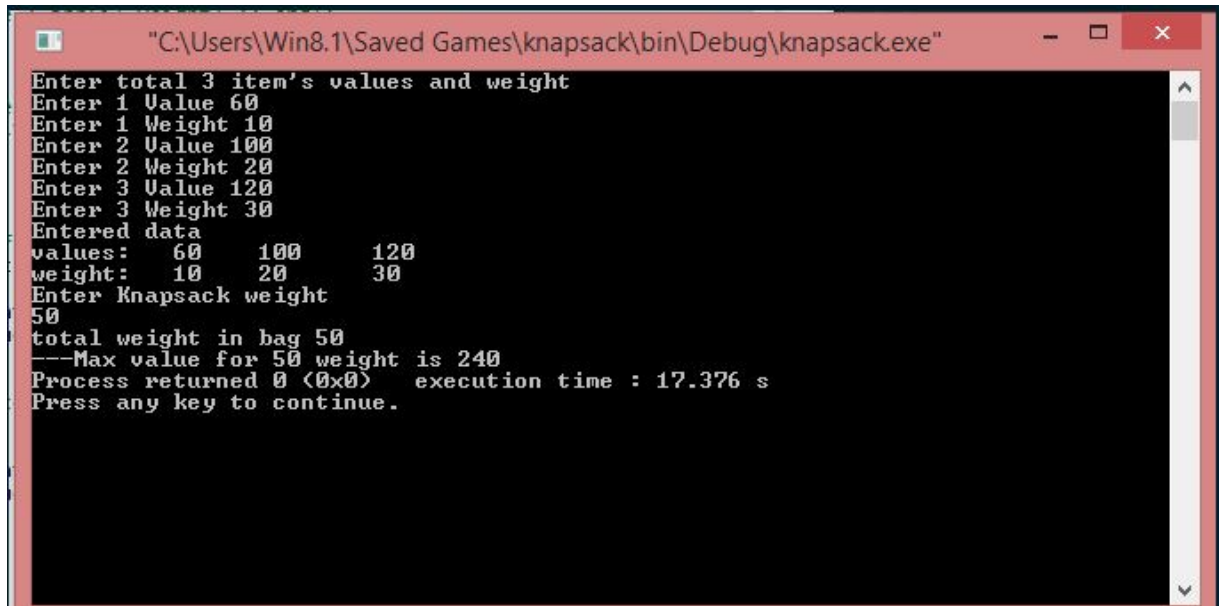
```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
typedef struct {
    int value;
    int weight;
    float density;
}Item;
void input(Item items[],int sizeOfItems){
    cout<< "Enter total "<<sizeOfItems<<" item's values and weight" <<endl;
    for(int i=0; i<sizeOfItems; i++){
        cout<< "Enter "<< i+1 << " Value ";
        cin>> items[i].value;
        cout<< "Enter "<< i+1 << " Weight ";
        cin>> items[i].weight;
    }
}
void display(Item items[],int sizeOfItems){
    cout<< "values: ";
    for(int i=0; i<sizeOfItems; i++){
        cout<< items[i].value << "\t";
    }
    cout<<endl<< "weight: ";
    for(int i=0; i<sizeOfItems; i++){
        cout<< items[i].weight<< "\t";
    }
    cout<<endl;
}
bool compare(Item i1, Item i2){
    return (i1.density> i2.density);
}
float knapsack(Item items[],int sizeOfItems, int W){
    float totalValue=0, totalWeight=0;
    for(int i=0; i<sizeOfItems; i++){
        items[i].density = items[i].value/items[i].weight;
    }
    sort(items, items+sizeOfItems,compare);
```

```

for(int i=0; i<sizeOfItems; i++){
if(totalWeight + items[i].weight<= W){
totalValue += items[i].value ;
totalWeight += items[i].weight;
    } else {
        int wt = W-totalWeight;
totalValue += (wt * items[i].density);
totalWeight += wt;
        break;
    }
}
cout<< "total weight in bag " <<totalWeight<<endl;
    return totalValue;
}
int main()
{
    int W;
    Item items[3];
    input(items,3);
    cout<< "Entered data \n";
    display(items,3);
    cout<< "Enter Knapsack weight \n";
    cin>> W;
    float mxVal = knapsack(items,3,W);
    cout<< "---Max value for "<< W <<" weight is "<<mxVal;
    return 0;
}

```


Output:



The screenshot shows a Windows command prompt window titled "C:\Users\Win8.1\Saved Games\knapsack\bin\Debug\knapsack.exe". The window contains the following text:

```
Enter total 3 item's values and weight
Enter 1 Value 60
Enter 1 Weight 10
Enter 2 Value 100
Enter 2 Weight 20
Enter 3 Value 120
Enter 3 Weight 30
Entered data
values:   60    100    120
weight:   10    20    30
Enter Knapsack weight
50
total weight in bag 50
---Max value for 50 weight is 240
Process returned 0 (0x0)   execution time : 17.376 s
Press any key to continue.
```

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 08	SEARCH FOR AN ELEMENT IN A TREE USING DIVIDE& CONQUER STRATEGY.
Date:	

Aim:

To write a program to search for an element in a tree using divide& conquer strategy.

Procedure:

Step 1: Start the Program

Step 2: Calculate the middle index mid using $(l + (r - l) / 2)$.

Step 3: Declare an array arr.

Step 4: Sort the array using sort.

Step 5: Call the Binary_search function.

Step 6: Display Result

Step 7: End of the Program

Program:

```
#include<bits/stdc++.h>
using namespace std;
int Binary_search(int arr[],int l,int r, int key)
{
    if(l<=r)
    {
        int mid = l+((r-l)/2);
        if(arr[mid]==key)
            return 1;
        else if(arr[mid]>key)
            return Binary_search(arr,l,mid-1,key);
        else
            return Binary_search(arr,mid+1,r,key);
    }
    return 0;
}
int main()
{
    int arr[]={3,4,5,6,7,8,4,5,2,8,9,13,14,17,19};
    int n=sizeof(arr)/sizeof(arr[0]);
    int key=79;
    sort(arr,arr+n);
    if(Binary_search(arr,0,n-1,key))
        cout<<"key is found in given array"<<endl;
    else
        cout<<"key is not found in given array"<<endl;
}
```

Output:

2 3 4 4 5 5 6 7 8 8 9 13 14 17 19

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 09	THE 8 QUEENS ON AN 8X8 MATRIX SO THAT NO TWO QUEENS ATTACK.
Date:	

Aim:

To write a program to place the 8 queens on an 8X8 matrix so that no two queens Attack.

Procedure:

Step 1: Start the Program

Step 2: Get the position of the first empty space (value of the rear variable)

Step 3: Assign the new value to position the rear in an array if the queue is not full.

Step 4: Increment the value of the rear variable

Step 6: Get the position of the first non-empty element (value of the front variable)

Step 7:Return the value of the element present at the index front if the queue is not empty

Step 8:Increment the value of the front variable

Step 9:End of the Program

Program:

```
#include<iostream>
using namespace std;
#define N 4
void printBoard(int board[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            cout<< board[i][j] << " ";
        cout<<endl;
    }
}
bool isValid(int board[N][N], int row, int col) {
    for (int i = 0; i < col; i++)
        if (board[row][i])
            return false;
    for (int i=row, j=col; i>=0 && j>=0; i--, j--)
        if (board[i][j])
            return false;
    for (int i=row, j=col; j>=0 && i<N; i++, j--)
        if (board[i][j])
            return false;
    return true;
}
bool solveNQueen(int board[N][N], int col) {
    if (col >= N)
        return true;
    for (int i = 0; i < N; i++) {
        if (isValid(board, i, col) ) {
            board[i][col] = 1;
            if ( solveNQueen(board, col + 1))
                return true;
            board[i][col] = 0;
        }
    }
    return false;
}
bool checkSolution() {
    int board[N][N];
    for(int i = 0; i<N; i++)
```

```
for(int j = 0; j<N; j++)
    board[i][j] = 0;
    if ( solveNQueen(board, 0) == false ) {
cout<< "Solution does not exist";
    return false;
    }
printBoard(board);
    return true;
}
int main() {
checkSolution();
}
```

Output:

```
1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
```

Result:

Thus, the given Program has been executed successfully and output verified

PYTHON LAB

Ex.No: 01	ELEMENTARY DATA ITEMS, LISTS, DICTIONARIES AND TUPLES
Date:	

Aim:

To write a programs using elementary data items, lists, dictionaries and tuples

Procedure:

Step 1: Start the Program

Step 2: Initialize an empty list l.

Step 3: Append the integer 5 and 10 to the list.

Step 4: Print the message and the current state of the list.

Step 5: Remove the last element from the list using the pop() method.

Step 6: Print the message and the current state of the list after popping an element.

Step 7 : Create an empty set s.

Step 8 : Add the integers 5 and 10 to the set.

Step 9 : Print the message and the current state of the set.

Step 10 : Remove the integer 5 from the set using the remove() method.

Step 11: Print the tuple and dictionaries.

Step 12: End of the Program

Program:

```
l = []
l.append(5)
l.append(10)
print("Adding 5 and 10 in list", l)
l.pop()
print("Popped one element from list", l)
print()
s = set()
s.add(5)
s.add(10)
print("Adding 5 and 10 in set", s)
s.remove(5)
print("Removing 5 from set", s)
print()
t = tuple(l)
print("Tuple", t)
print()
d = {}
d[5] = "Five"
d[10] = "Ten"
print("Dictionary", d)
del d[10]
print("Dictionary", d)
```

Output:

Adding 5 and 10 in list [5, 10]
Popped one element from list [5]
Adding 5 and 10 in set {10, 5}
Removing 5 from set {10}
Tuple (5,)
Dictionary {5: 'Five', 10: 'Ten'}
Dictionary {5: 'Five'}

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 02	CONDITIONAL BRANCH
Date:	

Aim:

To write a program using Conditional branch in python

Procedure:

Step 1: Start the Program

Step 2: Declare three parameters (a, b, and c).

Step 3: Check if the triangle is isosceles (at least two sides are equal).

Step 4: Sides cannot form a triangle, return 'not a triangle'.

Step 5: Print the result of classifying a triangle with sides

Step 6: Display Result

Step : End of the Program

Program:

```
def classify_triangle(a, b, c):
    if a + b > c and a + c > b and b + c > a:
        if a == b == c:
            return 'equilateral'
    elif a == b or a == c or b == c:
        return 'isosceles'
    else:
        return 'scalene'
    else:
        return 'not a triangle'
print(classify_triangle(3, 4, 5))
print(classify_triangle(3, 4, 6))
print(classify_triangle(3, 4, 7))
print(classify_triangle(3, 3, 5))
```

Output:

scalene
scalene
not a triangle
isosceles

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 03	LOOPS
Date:	

Aim:

To write a programs using loops.

Procedure:

Step 1: Start the Program

Step 2: Read for_loop() that uses a for loop to generate a list of squares from 1 to 10.

Step 3: Read while_loop() that uses a while loop to achieve the same result as for_loop().

Step 4: Read nested_loop() that uses nested loops to create a multiplication table (3x3 matrix).

Step 5: Print the result

Step 7 : End of the Program

Program:

```
def for_loop():
    squares = []
    for num in range(1, 11):
        squares.append(num ** 2)
    return squares

def while_loop():
    num = 1
    squares = []
    while num <= 10:
        squares.append(num ** 2)
        num += 1
    return squares

def nested_loop():
    result = []
    for i in range(1, 4):
        inner_list = []
        for j in range(1, 4):
            inner_list.append(i * j)
        result.append(inner_list)
    return result

print("for loop:", for_loop())
print("while loop:", while_loop())
print("nested loop:", nested_loop())
```

Output:

for loop: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
while loop: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
nested loop: [[1, 2, 3], [2, 4, 6], [3, 6, 9]]

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 04	FUNCTIONS
Date:	

Aim:

To write a programs using functions

.

Procedure:

Step 1: Start the Program

Step 2: Initialize a variable total to store the sum of the arguments.

Step 3: Use a for loop to iterate through each argument in args and add it to the total.

Step 4: Return the final total after iterating through all the arguments.

Step 5: Print the result

Step 6: End of the Program

Program:

```
def sum_all(*args):  
    total = 0  
    for num in args:  
        total += num  
    return total  
print("Sum:", sum_all(1, 2, 3))  
print("Sum:", sum_all(1, 2, 3, 4, 5))
```

Output:

Sum: 6
Sum: 15

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 05	EXCEPTION HANDLING
Date:	

Aim:

To write a programs using exception handling

.

Procedure:

Step 1: Start the Program

Step 2: Prompt the user to input their age and convert it to an integer.

Step 3: Check if the entered age is negative. If so, raise a ValueError with a specific message.

Step 4: Check if the age is less than 21. If so, raise a custom AgeRestrictionError with a specific message.

Step 5: Print a success message if no exceptions are raised.

Step 6: Display Result

Step : End of the Program

Program:

```
def apply_for_pg_program():
    try:
        age = int(input("Enter your age: "))

        if age < 0:
            raise ValueError("Age cannot be negative.")

        if age < 21:
            raise AgeRestrictionError("You must be at least 21 years old to apply for
the PG program.")

    print("Congratulations! You are eligible to apply for the PG program.")

    except ValueError as ve:
        print("Error: {}. Please enter a valid age.".format(ve))
    except AgeRestrictionError as are:
        print("Error: {}".format(are))
    except Exception as e:
        print("An unexpected error occurred: {}".format(e))

class AgeRestrictionError(Exception):
    pass

if __name__ == "__main__":
    apply_for_pg_program()
```

Output:

Enter your age: 22

Congratulations! You are eligible to apply for the PG program.

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 06	INHERITANCE
Date:	

Aim:

To write a programs using inheritance.

Procedure:

Step 1: Start the Program

Step 2: Create a class named BinarySearchTree to represent a Binary Search Tree.

Step 3: Create a insert method to add a node with the given data to the Binary SearchTree.

Step 4: Create a minNode method to find and return the node with the smallest value in a given subtree.

Step 5: Insert multiple elements into the BST using the insert method.

Step 6: Display Result

Step : End of the Program

Program:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("Subclasses must implement the 'speak'
method.")

class Dog(Animal):
    def speak(self):
        return "{} says Woof!".format(self.name)

class Cat(Animal):
    def speak(self):
        return "{} says Meow!".format(self.name)

class Bird(Animal):
    def speak(self):
        return "{} says Chirp!".format(self.name)

if __name__ == "__main__":
    dog = Dog("Buddy")
    cat = Cat("Whiskers")
    bird = Bird("Tweetie")

    print(dog.speak())
    print(cat.speak())
    print(bird.speak())
```

Output:

Buddy says Woof!

Whiskers says Meow!

Tweetie says Chirp!

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 07	POLYMORPHISM
Date:	

Aim:

To write a programs using polymorphism.

Procedure:

Step 1: Start the Program

Step 2: Define a base class Student with an `__init__` method to initialize name and student ID.

Step 3: Override the `__init__` method to include the major attribute.

Step 4: Define another subclass PostgraduateStudent similar to UndergraduateStudent.

Step 5: Override the `__init__` method to include the `research_topic` attribute.

Step 6: Call the `display_student_info` function to print the information of both student instances.

Step : End of the Program

Program:

```
class Student:
    def __init__(self, name, student_id):
        self.name = name
    self.student_id = student_id
    def display_info(self):
        raise NotImplementedError("Subclasses must implement the 'display_info'
method.")
class UndergraduateStudent(Student):
    def __init__(self, name, student_id, major):
        super().__init__(name, student_id)
    self.major = major
    def display_info(self):
        return "Undergraduate Student: {}, ID: {}, Major: {}".format(self.name,
self.student_id, self.major)
class PostgraduateStudent(Student):
    def __init__(self, name, student_id, research_topic):
        super().__init__(name, student_id)
    self.research_topic = research_topic

    def display_info(self):
        return "Postgraduate Student: {}, ID: {}, Research Topic:
{}".format(self.name, self.student_id, self.research_topic)

def display_student_info(student):
    print(student.display_info())

if __name__ == "__main__":
    undergrad_student = UndergraduateStudent("Gokul", "23UCS04", "Computer
Science")
    postgrad_student = PostgraduateStudent("Apsar", "23PCS02", "Machine
Learning")

display_student_info(undergrad_student)
display_student_info(postgrad_student)
```

Output:

Undergraduate Student: Gokul, ID: 23UCS04, Major: Computer Science

Postgraduate Student: Apsar, ID: 23PCS02, Research Topic: Machine Learning

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 08	FILE OPERATION
Date:	

Aim:

To write a programs using file operation in python

Procedure:

Step 1: Start the Program

Step 2: Open the file in write mode ('w') and write the provided content to the file

Step 3: Define a function read_from_file that takes a filename as a parameter.

Step 4: Open the file in read mode ('r') and read the content from the file.

Step 5: Call the write_to_file function to write data to the file.

Step 6: Call the read_from_file function to read and print the content of the file.

Step : End of the Program

Program:

```
def write_to_file(filename, content):
    try:
        with open(filename, 'w') as file:
            file.write(content)
        print("Data written to {} successfully.".format(filename))
    except Exception as e:
        print("Error writing to {}: {}".format(filename, e))

def read_from_file(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
        print("Content of {}: \n{}".format(filename, content))
    except FileNotFoundError:
        print("File {} not found.".format(filename))
    except Exception as e:
        print("Error reading from {}: {}".format(filename, e))

if __name__ == "__main__":

    filename = "example.txt"
    data_to_write = "Hello, this is an example!\nPython file operations."
    write_to_file(filename, data_to_write)
    read_from_file(filename)
```


Output:

Data written to example.txt successfully.

Content of example.txt:

Hello, this is an example!

Python file operations.

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 09	MODULE
Date:	

Aim:

To write a programs using module.

Procedure:

Step 1: Start the Program

Step 2: Create a separate script main_program.py that imports the calculator module.

Step 3: The main block of main_program.py, define two numbers (num1 and num2).

Step 4: Print the results of each operation.

Step 5: End of the Program

Program:

```
# calculator.py

def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    if y != 0:
        return x / y
    else:
        return "Cannot divide by zero"

# main_program.py
import calculator

if __name__ == "__main__":
    num1 = 10
    num2 = 5
    result_add = calculator.add(num1, num2)
    result_subtract = calculator.subtract(num1, num2)
    result_multiply = calculator.multiply(num1, num2)
    result_divide = calculator.divide(num1, num2)

    print("Addition: {}".format(result_add))
    print("Subtraction: {}".format(result_subtract))
    print("Multiplication: {}".format(result_multiply))
    print("Division: {}".format(result_divide))
```

Output:

Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0

Result:

Thus, the given Program has been executed successfully and output verified

Ex.No: 10	CREATING DYNAMIC AND INTERACTIVE WEBPAGES USING FORMS.
Date:	

Aim:

To write a programs using creating dynamic and interactive WebPages using forms.

Procedure:

Step 1: Start the Program

Step 2: The home page ('/'), rendering the 'index.html' template.

Step 3: Route for form processing ('/process_form') that accepts POST requests..

Step 4: Run the Flask app if the script is being executed directly.

Step 5: Create an 'index.html' template with a simple form for entering name and age.

Step 6: Create a 'result.html' template to display the submitted form data.

Step : End of the Program

Program:

#app.py

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/process_form', methods=['POST'])
def process_form():
    name = request.form['name']
    age = request.form['age']
    return render_template('result.html', name=name, age=age)

if __name__ == '__main__':
    app.run(debug=True)
```

#index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Dynamic Form Example</title>
</head>
<body>
<h1>Dynamic Form Example</h1>
<form action="/process_form" method="post">
<label for="name">Name:</label>
<input type="text" id="name" name="name" required><br>

<label for="age">Age:</label>
<input type="number" id="age" name="age" required><br>
```

```
<input type="submit" value="Submit">
</form>
</body>
</html>
```

#result.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Form Submission Result</title>
</head>
<body>
<h1>Form Submission Result</h1>
<p>Name: {{ name }}</p>
<p>Age: {{ age }}</p>
</body>
</html>
```

Result:

Thus, the given Program has been executed successfully and output verified